

Informe laboratorio Prisma ORM

1. Introducción

En el marco del proyecto de modernización de capacidades operativas del equipo de desarrollo, surge la necesidad de evaluar alternativas al ORM actualmente estandarizado, **Sequelize**. Debido a las limitaciones técnicas y problemas recurrentes detectados en el flujo de trabajo actual, este laboratorio explora las ventajas operativas de **Prisma ORM**. El objetivo es validar su viabilidad tecnológica mediante la prueba de funciones críticas y la exploración de futuras implementaciones que optimicen la mantenibilidad del software.

2. Problema

Sequelize presenta limitaciones al ser una herramienta antigua, entre ellas:

- Tooling menos flexible y poco amigable con ESM moderno
- Integración limitada y poco estricta con TypeScript
- Manejo de migraciones y seeders menos robusto
- Mayor fricción al escalar o mantener proyectos con arquitecturas modernas

Estas limitaciones motivaron la evaluación y migración hacia una alternativa más actual que permita mayor control, mantenibilidad y claridad en el desarrollo.

3. Rúbricas

Para este laboratorio, se decidió realizar un servicio web pequeño de autenticación basado en usuarios y un recurso que cada usuario pueda acceder al iniciar sesión: Logros.

Con este pequeño servicio, se pusieron a prueba las siguientes rúbricas **en comparación con Sequelize**:

- Complejidad de Instalación y configuración.
- Creación de modelos (o entidades).
- Asociación de modelos (o entidades).
- Consultas a la entidad.
- Migraciones.
- Testing.

4. Principales observaciones

4.1 Ecosistema: TypeScript vs. JavaScript

Una de las diferencias fundamentales entre ambas herramientas radica en su arquitectura de diseño y prioridades de desarrollo:

- **Sequelize:** Históricamente diseñado para un ecosistema JavaScript, lo que dificulta la integración fluida de tipos robustos sin configuraciones adicionales complejas o propensas a errores.
- **Prisma:** Concebido como un ORM moderno que prioriza la seguridad de tipos (*Type Safety*). Su motor genera un cliente a medida basado en el esquema de la base de datos, garantizando una experiencia de "extremo a extremo" (*end-to-end type safety*).

Beneficios Clave: Prisma sobre Sequelize

La transición hacia un modelo basado en TypeScript con Prisma ofrece mejoras sustanciales en la Experiencia de Desarrollo (DX):

1. **Tipado Automático:** A diferencia de JS, donde los errores de estructura se detectan en ejecución, Prisma permite que el editor de código comprenda la base de datos de forma nativa, ofreciendo un autocompletado preciso.
2. **Seguridad en Tiempo de Compilación:** Reduce drásticamente los errores en producción al validar que las consultas coincidan exactamente con el esquema definido antes de ejecutar el código.
3. **Refactorización Segura:** Facilita la modificación del esquema de datos; cualquier cambio impacta inmediatamente en el tipado, señalando errores de referencia en todo el proyecto de manera automática.
4. **Mantenibilidad:** La estructura clara de Prisma reduce la carga cognitiva del desarrollador al interactuar con modelos complejos.

4.2 Instalación y configuración

Aunque la arquitectura de archivos de Prisma puede parecer compleja inicialmente debido a que gestiona sus propios artefactos (como el archivo `schema.prisma`), el proceso de instalación es altamente automatizado y guiado mediante su CLI. A diferencia de Sequelize, que depende de archivos de configuración más manuales, Prisma centraliza la fuente de verdad en el esquema.

Un punto de convergencia entre ambos es la necesidad de una instancia inicial para establecer la comunicación con el motor de datos. Sin embargo, la implementación difiere notablemente:

- **Sequelize:** Suele configurarse mediante un objeto de opciones que separa el host, puerto, base de datos y credenciales.
- **Prisma:** Estandariza la conexión mediante el uso de una **Connection URL** (Connection String). Esta URL puede ser la provista por el entorno de despliegue o una definida localmente, pero es el método mandatorio para la inicialización del cliente.

Configuración Sequelize

En Sequelize, la conexión se gestiona instanciando el objeto con parámetros granulares y ejecutando un método de sincronización o autenticación manual:

```

11 const sequelize = new Sequelize(DB_NAME, DB_USER, DB_PASSWORD, {
12   host: DB_HOST,
13   port: Number(DB_PORT),
14   dialect: 'postgres',
15   logging: Boolean(DEBUG),
16   pool: {
17     max: 7,
18     min: 0,
19     acquire: 30000,
20     idle: 10000,
21   },
22 });
23
24 const connectDB = async () => {
25   try {
26     await import('#models/index');
27     await sequelize.sync({ alter: true });
28     console.log('✅ Conexión a la base de datos establecida correctamente');
29   } catch (error) {
30     console.error('❌ Error al conectar a la base de datos: ', error);
31     process.exit(1);
32   }
33 };
34
35 export { sequelize, connectDB };
36

```

Configuración Prisma

Prisma utiliza un enfoque basado en adaptadores (en este caso pg) y una instancia de clase única (Singleton) que se conecta de manera asíncrona:

```

6   const adapter = new PrismaPg({ connectionString: DATABASE_URL });
7
8   /**
9    * Instancia global (Singleton) del cliente de Prisma.
10   * Utiliza esta instancia para realizar todas las operaciones CRUD.
11   */
12   export const prisma = new PrismaClient({ adapter });
13
14   /**
15   * Inicializa la conexión con la base de datos a través de Prisma Client.
16   * Esta función instancia una conexión utilizando el adaptador configurado
17   * y registra el resultado de la operación en los logs del sistema.
18   * @example
19   * // Uso en el punto de entrada de la aplicación (ej. app.ts)
20   * await connectDb();
21   */
22   export const connectDb = async () => {
23     try {
24       await prisma.$connect();
25       logger.info('Database connections established');
26     } catch (error) {
27       logger.error('Error establishing database connection', error);
28     }
29   };
30

```

4.3 Creación de entidades

La gestión de entidades presenta un cambio de paradigma en la organización del proyecto:

- **Sequelize:** Emplea un enfoque distribuido donde cada entidad se define en un archivo individual. Este esquema requiere un archivo central (comúnmente index.js o models.js) encargado de importar, inicializar y establecer manualmente las relaciones (asociaciones) entre todos los modelos. Esto puede derivar en una gestión compleja a medida que el número de tablas aumenta.
- **Prisma:** Adopta un enfoque declarativo y centralizado mediante el archivo schema.prisma. En este único archivo se define tanto la configuración del motor de datos como la estructura de las entidades y sus relaciones. Esta centralización mejora la visibilidad global del esquema y facilita la comprensión del modelo de datos para cualquier desarrollador.

Modelos en Prisma

Prisma utiliza una sintaxis propia muy legible que permite definir tipos, restricciones y relaciones de forma compacta:

```
prisma > schema.prisma > ProductCompability
Generate
1 generator client {
2   provider = "prisma-client"
3   output   = "../generated/prisma"
4 }
5
6 datasource db {
7   provider = "postgresql"
8 }
9
10 model User {
11   id        Int        @id @default(autoincrement())
12   email     String      @unique
13   name      String?
14   password  String
15   role      Role        @default(USER)
16   createdAt DateTime    @default(now())
17   updatedAt DateTime    @default(now()) @updatedAt
18   active    Boolean     @default(true)
19   sessions  Session[]
20 }
21
```

Modelos Sequelize

En contraste, Sequelize requiere la exportación individual y el mapeo manual de asociaciones, lo que incrementa el volumen de código *boilerplate*:

```
const City = sequelize.define(
  'City',
  {
    id: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true,
      autoIncrementIdentity: true,
    },
    name: {
      type: DataTypes.STRING,
      allowNull: false,
    },
    cityCode: {
      type: DataTypes.STRING,
      unique: true,
      allowNull: false,
    },
  },
  {
    timestamps: true,
    defaultScope: {
      attributes: {
        exclude: ['createdAt', 'updatedAt'],
      },
    },
  }
);
```

4.4 Asociación de entidades

La gestión de entidades presenta un cambio de paradigma en la organización del proyecto:

- **Sequelize:** Emplea un enfoque distribuido donde cada entidad se define en un archivo individual. Este esquema requiere un archivo central (comúnmente `index.js` o `models.js`) encargado de importar e inicializar los modelos.
- **Prisma:** Adopta un enfoque declarativo y centralizado mediante el archivo `schema.prisma`. En este único archivo se define tanto la configuración del motor de datos como la estructura de las entidades.

Asociaciones Sequelize

Sequelize (Enfoque Imperativo): Las relaciones se definen mediante lógica de programación fuera de la definición de la tabla (usando métodos como `hasMany`, `belongsTo`, etc.). Si bien ofrece flexibilidad, esta metodología tiende a volverse extensa y difícil de seguir en esquemas con múltiples relaciones cruzadas, obligando al desarrollador a saltar entre archivos para entender el grafo de datos.

```

export { default as City } from './cities.model.js';
export { default as State } from './state.model.js';
export { default as Action } from './action.model.js';
export { default as Zone } from './zones.model.js';

// Relaciones
const FK_REQUIRED = { allowNull: false };
const CASCADE = { onDelete: 'CASCADE', onUpdate: 'CASCADE', hooks: true };

// Usuario - Roles
Role.hasMany(User, {
  foreignKey: { name: 'RoleId', ...FK_REQUIRED },
  ...CASCADE,
});
User.belongsTo(Role, {
  foreignKey: { name: 'RoleId', ...FK_REQUIRED },
  ...CASCADE,
});

```

Asociaciones Prisma

Prisma (Enfoque Declarativo): Las relaciones se definen directamente dentro del bloque de cada modelo en el esquema. Prisma utiliza relaciones virtuales para facilitar la navegación en el código y relaciones físicas para la integridad de la base de datos. Este método es más legible, ya que la estructura completa de la tabla y sus dependencias son visibles en un solo lugar.

```

model Vehicle {
  id          Int           @id @default(autoincrement())
  brandId     Int
  model       String
  year        Int
  displacement Int?
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @default(now()) @updatedAt
  active      Boolean        @default(true)
  compatibilities ProductCompatibility[]
  brand       Brand          @relation(fields: [brandId], references: [id])

  @@unique([brandId, model, year])
}

model Brand {
  id          Int           @id @default(autoincrement())
  name        String
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @default(now()) @updatedAt
  active      Boolean        @default(true)
  slug        String        @unique
  products    Product[]
  vehicles    Vehicle[]
}

```

4.5 Consulta de entidades

Aunque la lógica para consultar datos es conceptualmente similar en ambos ORMs, la implementación en Prisma ofrece una interfaz más intuitiva y robusta:

- **Sequelize:** El acceso a los métodos de consulta se realiza a través de la clase del modelo importado (ej. `User.findAll()`). Aunque es funcional, la falta de un tipado fuerte nativo obliga al desarrollador a conocer de antemano la estructura de retorno o definir interfaces manuales para evitar errores de acceso a propiedades inexistentes.
- **Prisma:** Las consultas se ejecutan a través de la instancia del cliente centralizado (ej. `prisma.user.findMany()`). La ventaja crítica aquí es que los resultados están **automáticamente tipados** según el esquema definido. Esto significa que si se realiza una consulta, el IDE sugerirá únicamente los campos existentes en esa tabla, eliminando errores comunes de "undefined" en tiempo de ejecución.

Consultas sequelize

Requiere importar el modelo específico y, en muchos casos, manejar la lógica de negocio mezclada con la de persistencia:

```
> /** ...
export const create = async (data) =>
  User.create({ ...data, password: await hashValue(data.password) })
> /** ...
export const findAll = () => User.findAll()
```

Consultas Prisma

Se beneficia de una sintaxis más clara ("Fluent API") y un soporte nativo para operaciones en lote o filtros complejos con validación inmediata:

```
export const create = (data: NameSchema) =>
  !Array.isArray(data) &&
  prisma.category.create({ data: { ...data, slug: slug(data.name) } });
> /** ...
> export const createMany = async (data: NameSchema[]) => { ...
};
> /** ...
export const findMany = () =>
  prisma.category.findMany({ where: { active: true } });
```

4.6 Migraciones y sincronización

La gestión de cambios en la estructura de la base de datos es una de las mayores diferencias operativas entre ambas herramientas:

- **Sequelize (Manejo Manual e Imperativo):** Aunque Sequelize posee un sistema de migraciones, este se percibe desactualizado para estándares modernos. Requiere que el desarrollador escriba manualmente archivos de migración detallando cada cambio (métodos up y down). Además, el uso persistente de require (CommonJS) en su CLI dificulta la integración en proyectos que utilizan módulos de ES o TypeScript de forma nativa, aumentando la fricción técnica.
- **Prisma (Manejo Automatizado y Declarativo):** Prisma revoluciona este flujo mediante Prisma Migrate. El desarrollador simplemente modifica el archivo schema.prisma y ejecuta un comando (ej. npx prisma migrate dev). La herramienta compara el esquema con el estado actual de la base de datos y genera automáticamente el archivo SQL necesario. Este sistema no solo es más rápido, sino que reduce el error humano al garantizar que el código y la base de datos estén siempre sincronizados.

4.7 Testing

Para este laboratorio, se utilizó **Vitest** como motor de pruebas (*test runner*) para comparar la facilidad de implementación de pruebas unitarias:

- **Testing en Sequelize:** Se basa en la creación de "espías" (*spies*) sobre las dependencias del modelo. Es un proceso relativamente directo pero propenso a errores, ya que requiere simular respuestas de métodos estáticos de clases importadas, lo que a menudo rompe el tipado si no se gestiona con interfaces personalizadas.
- **Testing en Prisma:** Aunque requiere un paso adicional de configuración para "mockear" la instancia global del PrismaClient, Prisma ofrece soporte oficial y patrones claros en su documentación. El uso de librerías como vitest-mock-extended permite realizar un **Deep Mock** de toda la instancia del cliente, facilitando la simulación de consultas complejas (consultas, migraciones y modelos) de manera centralizada y manteniendo la integridad de los tipos en los tests.

```
/**
 * Las variables usadas dentro de `vi.mock` DEBEN empezar con la palabra "mock".
 * Vitest las elevará automáticamente después de los imports, evitando el ReferenceError.
 */
export const mockPrisma = mockDeep<PrismaClient>();

vi.mock('#config/db', () => ({
  prisma: mockPrisma,
  default: mockPrisma,
}));

export const prismaMock = mockPrisma as unknown as DeepMockProxy<PrismaClient>;
```


5. Análisis de Transición y Curva de Aprendizaje

La migración de Sequelize a Prisma no es solo un cambio de herramienta, sino un cambio estructural en el flujo de trabajo del equipo de Backend. Se han identificado los siguientes desafíos críticos:

- **Evolución del Lenguaje (JS a TS):** El paso de JavaScript a TypeScript es el cambio más significativo. Aunque implica una curva de aprendizaje inicial, es el pilar que sostiene la seguridad del código y reduce drásticamente la deuda técnica a largo plazo.
- **Paradigma de Modelado:** La transición de un sistema de archivos distribuidos (un archivo por modelo) a un esquema centralizado (schema.prisma) requiere que el equipo se adapte a una nueva forma de visualizar la arquitectura de la base de datos.
- **Arquitectura de Navegación:** El nuevo manejo de carpetas, artefactos generados automáticamente y la lógica de migraciones representa un cambio complejo. No obstante, dada la capacidad técnica actual del equipo, este reto es superable y se compensa con la eficiencia operativa ganada tras la adaptación.

6. Conclusión

La adopción de Prisma representa un salto cualitativo en la robustez del código. La capacidad de detectar inconsistencias antes del despliegue, junto con un sistema de migraciones automatizado, una arquitectura de pruebas sólida y un tipado de extremo a extremo, asegura una base de código más estable, mantenible y fácil de escalar en comparación con las prácticas actuales basadas en Sequelize. El costo de la curva de aprendizaje se justifica plenamente por la mejora sustancial en la calidad del software entregado.